C# Ders - 16 Ekim 2026 Ders Notları

1. Döngüler: `while` Döngüsü ve Konsol Menü Mimarisi

`while` döngüsü, parantez içindeki mantıksal koşul `true` olduğu sürece bloğunu tekrar tekrar çalıştırır. Çalışma sayısı önceden kesin belli olmayan, bir durum bayrağına (flag) veya kullanıcı girdisine bağlı senaryolarda tercih edilir.

A. Temel Sayaçlı `while`

C#

```
int count = 0;
while (count < 10)
{
    Console.WriteLine("Döngü turu: " + count);
    count++; // Sayacı artırmazsan sonsuz döngü (infinite loop) oluşur!
}
```

B. Durum Bayrağı (Flag) ile İnteraktif Menü Mimarisi

Kodundaki manav panelinde uygulanan kalıp, konsol tabanlı otomasyonlarda temel standarttır:

- `bool is_online = true;` bayrağı tanımlanır.
- `while (is_online)` döngüsü başlatılır.
- Kullanıcı `99` veya çıkış komutu verene kadar menü ekranda dönmeye devam eder; çıkış durumunda `is_online = false;` yapılarak döngü kontrollü sonlandırılır.

C#

```
bool is_online = true;
while (is_online)
{
```

```
    Console.WriteLine("[1] İşlem, [2] Çıkış");
    int.TryParse(Console.ReadLine(), out int secim);

    switch (secim)
    {
        case 1:
            // İşlem kodları...
            break;
        case 2:
            is_online = false; // Döngüyü kırar
            break;
        default:
            Console.WriteLine("Geçersiz seçim!");
            break;
    }
}
```

C. Deneme Hakkı Sınırlaması (Login / Brute-Force Önleme)

Kullanıcıya belirli sayıda hak tanırken `while` ve sayaç kombinasyonu kullanılır:

C#

```
int userTry = 0;
while (userTry < 3)
{
    // Giriş kontrolü...
    if (basarili)
    {
        userTry = 3; // veya 'break;' ile döngüden çıkılır
    }
    else
    {
        userTry++;
        Console.WriteLine($"Kalan Hak: {3 - userTry}");
    }
}
```

2. İleri Düzey `string` Manipülasyonu

Metinler (`string`), C#'ta referans tiplidir ve **değiştirilemezdir (immutable)**. Bir metot çağrıldığında orijinal metin değişmez; bellekte yeni bir metin üretilir.

A. Arama, Sorgulama ve Sınır Kontrolleri

- `.Length` **(Property):** Metnin toplam karakter sayısını döner.
- `.Contains("metin")`**:** İlgili ifadenin metin içinde geçip geçmediğini (`bool`) sorgular.
- `.StartsWith("İ")`**:** Metnin belirtilen karakter/kelime ile başlayıp başlamadığını kontrol eder.
- `.EndsWith("!")`**:** Metnin belirtilen karakter/kelime ile bitip bitmediğini kontrol eder.
- `.IndexOf("E")`**:** Aranan ifadenin metinde **ilk görüldüğü** indeks numarasını (`int`) döner (Bulamazsa `-1` döner).
- `.LastIndexOf("e")`**:** Aranan ifadenin metinde **en son görüldüğü** indeks numarasını döner.

B. Parçalama, Kesme ve Birleştirme

- `.Split(' ')`**:** Metni verilen karaktere göre böler ve bir `string[]` dizisi döner.C#

```
string[] kelimeler = metin.Split(" ");
```

- `.Substring(baslangic, uzunluk)`**:** Belirtilen indeksten başlayarak istenen karakter adedi kadar parçayı çeker.C#

```
string parca = metin.Substring(0, 15); // 0. indeksten başla, 15 karakter al
```

- `.Remove(baslangic)` **/** `.Remove(baslangic, adet)`**:** Belirtilen indeksten sonrasını veya belirli bir aralığı kırparak metinden çıkarır.C#

```
yazar.Remove(6);      // 6. indeksten sonrasını komple siler
yazar.Remove(0, 6);   // 0'dan başla, 6 karakter sil (baştan keser)
```

- `string.Concat(...)`**:** Verilen metin parçalarını araya ayraç koymadan uç uca ekler.
- `string.Join(ayrac, dizi)`**:** Dizi elemanlarını aralarına belirlenen ayracı (`", "`, `"-"` vb.) koyarak tek bir `string` haline getirir.C#

```
string liste = string.Join(", ", kelimeler);
```

C. Değiştirme (Replace) ve Slug/URL Üretimi

- `.Replace(eski, yeni)`: Metin içinde bulunan tüm eski karakter/kelimeleri yenisiyle değiştirir. Zincirleme (chaining) olarak kullanılabilir:C#

```
string url = metin.ToLower()
                  .Replace(" ", "-")
                  .Replace("ı", "i")
                  .Replace("â", "a")
                  .Replace("'", "")
                  .Replace("ü", "u")
                  .Replace("!", "");
```

D. Boşluk Temizleme (Sanitization)

- `.Trim()`: Metnin hem başındaki hem sonundaki boşlukları temizler.
- `.TrimStart()`: Sadece metnin başındaki (soldaki) boşlukları temizler.
- `.TrimEnd()`: Sadece metnin sonundaki (sağdaki) boşlukları temizler.

E. Eşitlik Kontrolü ve Kültür Duyarlılığı

- `.Equals(...)`: İki metni kıyaslar.C#

```
string yazi = "ABC";
string yazi2 = "abc";

yazi.Equals(yazi2); // false (Büyük/küçük harf duyarlı)
yazi.Equals(yazi2, StringComparison.OrdinalIgnoreCase); // true (Büyük/küçük harf far
kını yok sayar)
```

F. Boşluk & Null Denetimleri

- `string.IsNullOrEmpty(deger)`: Değer `null` veya boş string (`""`) ise `true` döner.
- `string.IsNullOrWhiteSpace(deger)`: Değer `null`, `""` VEYA **yalnızca boşluk karakterlerinden (`" "`)**oluşuyorsa `true` döner. Kullanıcı giriş doğrulamasında

en güvenli kontroldür.

G. Karakter Dizisine Dönüştürme

- `.ToCharArray()`: Metni `char[]` dizisine ayırır; karakterler üzerinde döngüyle tek tek gezinmeyi sağlar.C#

```
char[] harfler = yazar.ToCharArray();
```

3. Bellek Performansı: `StringBuilder`

C#'ta `string` nesneleri **değiştirilemez (immutable)** olduğu için `+=` ile yapılan her ekleme, RAM'de (Heap) yeni bir string nesnesi tahsis eder ve eskisini çöpe atar. Çok fazla metin birleştirilen döngülerde Garbage Collector yorulur.

`StringBuilder` **(System.Text kütüphanesinde yer alır):**

- Metni dinamik bir bellek tamponunda (buffer) tutar; aynı nesne üzerinde değişiklik yapar (**mutable**).
- `.Append(...)`: Mevcut metnin sonuna ekleme yapar.
- `.Length`: İçerisindeki metnin mevcut karakter sayısını verir.
- `.Capacity`: Bellekte tahsis edilmiş toplam kapasiteyi belirtir (Metin sığmadıkça kapasite otomatik olarak 2 katına çıkarılır).

C#

```
using System.Text;

var metin = new StringBuilder("Başlangıç metni");
metin.Append(" - Ek 1");
metin.Append(" - Ek 2");

Console.WriteLine(metin.ToString()); // Nihai string çıktısı alınır
```

4. Matematiksel Fonksiyonlar: `Math` Sınıfı

`Math` sınıfı, `System` isim alanı altında bulunan statik bir sınıftır; doğrudan çağrılır.

Sabitler (Constants)

- `Math.PI`: ππ sayısı (3.14159...3.14159...)
- `Math.E`: Euler sabiti (2.71828...2.71828...)
- `Math.Tau`: ττ sabiti (2π≈6.28318...2π≈6.28318...)

Fonksiyonlar

Aa Title	Metot	Görevi	Örnek	# Sonuç	Files
Untitled	`Math.Abs(x)`	Mutlak değer alır.	`Math.Abs(-89)`	89	
Untitled	`Math.Pow(taban, us)`	Üs alma işlemi (xy).	`Math.Pow(5, 3)`	125	
Untitled	`Math.Sqrt(x)`	Karekök hesaplar (x).	`Math.Sqrt(64)`	8	data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg" width="400em" height="1.08em" viewBox="0 0 400000 1080" preserveAspectRatio="xMinYMin slice"><path d="M95,702c-2.7,0,-7.17,-2.7,-13.5,-8c-5.8,-5.3,-9.5

Title	Metot	Görevi	Örnek	Sonuç	Files
					,-10,-9.5,-14c0,-2,0.3,-3.3,1,-4c1.3,-2.7,23.83,-20.7,67.5,-54c44.2,-33.3,65.8,-50.3,66.5,-51c1.3,-1.3,3,-2,5,-2c4.7,0,8.7,3.3,12,10s173,378,173,378c0.7,0,35.3,-71,104,-213c68.7,-142,137.5,-285,206.5,-429c69,-144,104.5,-217.7,106.5,-221l0 -0c5.3,-9.3,12,-14,20,-14H400000v40H845.2724s-225.272,467,-225.272,467s-235,486,-235,486c-2.7,4.7,-9,7,-19,7c-6,0,-10,-1,-12,-3s-194,-422,-194,-422s-65,47,-65,47zM834 80h400000

Aa Title	Metot	Görevi	Örnek	# Sonuç	Files
					v40h-400000z"></path></svg>
Untitled	`Math.Min(a, b)`	İki sayıdan küçük olanı döner.	`Math.Min(45, 78)`	45	
Untitled	`Math.Max(a, b)`	İki sayıdan büyük olanı döner.	`Math.Max(45, 78)`	78	
Untitled	`Math.Floor(x)`	Daima **aşağıya** (küçük tam sayıya) yuvarlar.	`Math.Floor(4.89)`	4	
Untitled	`Math.Ceiling(x)`	Daima **yukarıya** (büyük tam sayıya) yuvarlar.	`Math.Ceiling(4.12)`	5	
Untitled	`Math.Truncate(x)`	Ondalık kısmı tamamen atar, sadece tam kısmı alır.	`Math.Truncate(4.89)`	4	
Untitled	`Math.Round(x)`	En yakın tam sayıya yuvarlar (Banker's rounding).	`Math.Round(4.89)`	5	

5. Rastgele Sayı Üretimi: `Random` Sınıfı

Sistem saatini tohum (seed) alarak rastgele değerler üreten sınıftır.

C#

```
Random rnd = new Random();

int rastgeleTamSayi = rnd.Next(1000);        // 0 ile 999 arasında (1000 dahil değil)
int aralikliSayi    = rnd.Next(100, 1000);   // 100 ile 999 arasında
double rastgeleOndalik = rnd.NextDouble();   // 0.0 ile 1.0 arasında rastgele sayı
```

6. Modern C# Sözdizimi: Raw String Literals (`"""`)

C# 11 ile tanıtılan **Raw String Literals**, üç veya daha fazla çift tırnak (`"""`) ile açılıp kapatılır.

- İçerisinde kaçış karakteri (`\n` , `\"` , `\\`) kullanma zorunluluğunu ortadan kaldırır.
- Satır sonlarını, girintileri (indentation) ve tırnak işaretlerini olduğu gibi korur.

C#

```
string metin = """
               Merhaba
               Bugün hava
               "yağmurlu"
               ankara'da
               """;
Console.WriteLine(metin);
```

7. Özet Algoritma İncelemesi: `while` ile Dinamik Palindrom Sayı Kontrolü

Kodundaki matematiksel basamak tersleme mantığı:

C#

```
int number = 121;
int original = number;
int tersi = 0;

while (number > 0)
{
    tersi = (tersi * 10) + (number % 10);
    number /= 10;
}
```

```
bool isPalindrom = (original == tersi);
```

1. `number % 10` ile son basamak yakalanır.
2. `tersi * 10` yapılarak önceki basamaklar sola kaydırılır ve yeni basamak eklenir.
3. `number /= 10` ile son basamak sayıdan düşülür.
4. Sayı 0 olana kadar döngü devam eder.